

Unidad 02 - Conceptos básicos sobre sistemas operativos

1. Definición de Sistema Operativo

Un sistema operativo es un software esencial y complejo que controla el hardware y facilita la interacción del usuario con la computadora. Funciona como intermediario entre el hardware (procesador, memoria, dispositivos) y los programas de usuario, proporcionando un entorno estable y seguro donde pueden ejecutarse aplicaciones. Sin un sistema operativo, un usuario tendría que interactuar con el hardware directamente mediante instrucciones de bajo nivel, lo que sería ineficiente y poco práctico. Entre sus funciones está gestionar recursos físicos y lógicos, coordinar tareas, proporcionar seguridad y simplificar el uso de la computadora.

2. Tipos de Sistemas Operativos

Por número de usuarios

- Monousuario: Solo permite trabajar a un usuario en un momento dado (ejemplo: MS-DOS). Estos sistemas son sencillos, pero limitan la colaboración y la productividad en entornos multiusuario.
- Multiusuario: Permiten que varios usuarios trabajen simultáneamente en el mismo sistema, gestionando el acceso a recursos para evitar conflictos. Muy comunes en redes y servidores (ejemplo: UNIX, GNU/Linux, Windows Server).

Por número de tareas

- Monotarea: Ejecuta una única tarea a la vez. Históricamente, este fue el diseño más sencillo, pero ineficiente.

- **Multitarea:** Gestiona varias tareas “al mismo tiempo” compartiendo recursos y alternando la CPU entre procesos (ejemplo: Windows moderno, Linux). Puede ser multitarea cooperativa o fácil, según la administración de la CPU.

Por número de procesadores

- **Monoprocesador:** Diseñados para ordenadores con una sola CPU.
- **Multiprocesador:** Distribuyen tareas entre varias CPUs, logrando mayor eficiencia, paralelismo y tolerancia a fallos (ejemplo: grandes servidores).

Por tipo de interfaz de usuario

- **CLI (Command Line Interface):** Interfaz por línea de comandos, requiere conocimientos avanzados de sintaxis (ejemplo: MS-DOS, Terminal de UNIX).
- **GUI (Graphical User Interface):** Interfaz gráfica, amigable y visual (ejemplo: Windows, macOS, GNOME en Linux), mejorando la experiencia para usuarios no expertos.

Por finalidad o uso específico

- **Sistemas de red:** Proveen capacidades avanzadas de comunicación y administración de recursos compartidos (archivos, impresoras) en una red local o extendida.
- **Sistemas distribuidos:** Coordinan varios ordenadores en red para trabajar como un sistema único, optimizando cargas y recursos.
- **Sistemas en tiempo real:** Responden a estímulos en tiempos muy precisos, esenciales en aplicaciones industriales o médicas (VxWorks, RTLinux).

Por forma de trabajo y tiempo de respuesta

- **Centralizado:** Recursos y gestión en un solo equipo.
- **Distribuido:** Recursos y gestión distribuidos en varias máquinas interconectadas.
- **Tiempo real:** Las respuestas del sistema ocurren dentro de un límite predictivo (automatización, control de maquinaria).

3. Funciones de un Sistema Operativo

- **Gestión de procesos:** Controla el ciclo de vida de los procesos: creación, ejecución, suspensión y finalización. Utiliza algoritmos de planificación para repartir la CPU eficientemente.
- **Gestión de memoria:** Supervisa la memoria RAM, decide qué procesos deben recibir memoria y cuándo, optimizando su uso y evitando interferencias.
- **Gestión de archivos:** Organiza el almacenamiento, acceso, modificación y eliminación de información en dispositivos de almacenamiento, imponen jerarquía y privilegios sobre archivos y carpetas.
- **Gestión de E/S:** Coordina la comunicación entre dispositivos externos (teclado, mouse, impresoras, discos) y los programas.
- **Gestión de almacenamiento secundario:** Controla el uso de discos duros, SSDs, dispositivos extraíbles.
- **Sistema de protección y seguridad:** Limita el acceso a recursos por parte de usuarios y programas, protege el sistema ante intrusos y errores.
- **Sistema de comunicación:** Permite la transmisión de datos entre procesos, usuarios, sistemas y redes.
- **Intérprete de órdenes (shell):** Recibe e interpreta comandos del usuario, facilitando la interacción mediante scripts o interfaces gráficas.
- **Gestión de errores:** Detecta y maneja distintos tipos de errores de software y hardware, garantizando la estabilidad del sistema.

4. Arquitectura de un SO

- **Núcleo (kernel):** Es el núcleo central encargado de gestionar los recursos básicos y coordinar la comunicación entre software y hardware. Está formado por subsistemas para procesos, memoria, almacenamiento, seguridad y dispositivos.
- **Capas y módulos:** Arquitecturas modernas dividen sus funciones en capas (cada una depende de la anterior) o módulos que pueden cargarse y descargarse dinámicamente (mejor mantenimiento y escalabilidad).

- Espacio del núcleo vs espacio de usuario: Separación fundamental para evitar que los errores de aplicaciones afecten al sistema completo.

5. Tipos de Kernel

- Monolítico: Todo el código esencial corre en el espacio del núcleo. Es rápido, pero menos seguro si hay errores (ejemplo: Linux clásico, UNIX).
- Por capas/anillos: Modulariza el kernel en capas, donde cada nivel provee servicios a la superior. Favorece la seguridad y la organización (ejemplo: THE, Windows NT).
- Microkernel: Solo ejecuta lo esencial, el resto de servicios funciona en espacio de usuario como procesos independientes (ejemplo: MINIX, QNX). Mejora la seguridad y el aislamiento, pero a veces implica más sobrecarga de comunicación.
- Máquina virtual: Proporciona varias máquinas completamente virtualizadas sobre un hardware físico, cada una puede ejecutar su propio SO aisladamente (ejemplo: IBM VM/370).
- Cliente-servidor: Arquitecturas modernas dividen el sistema en servidores (que proveen funciones) y clientes (que las usan), facilitando la modularidad.

6. Evolución histórica de los SO

- Primeros sistemas (años 40-50): No existían SO propiamente dichos, la programación era directa con tarjetas perforadas o cables conectados a mano.
- Sistemas por lotes (batch): Permiten procesar muchos trabajos automáticamente sin intervención manual, aumentando la eficiencia.
- Multiprogramación: Aparece la posibilidad de ejecutar varios programas en memoria a la vez, alternando recursos y mejorando rendimiento.
- Tiempo compartido: Permite que varios usuarios interactúen con la máquina en tiempo casi real, usando terminales remotas (ejemplo: Multics, UNIX).
- Sistemas distribuidos y multiprocesador: Aparecen con la proliferación de redes y equipos conectados, gestionando recursos múltiples y coordinando tareas intercambiables y redundantes para mayor eficiencia y tolerancia a fallos.

7. Tipos de licencias de software

- Privativas: Impiden ver, modificar o redistribuir el código fuente. Suelen asociarse a un contrato restrictivo de uso. Ejemplo: Microsoft Windows, Adobe Photoshop. Ventaja: soporte incluido; desventaja: dependencia del fabricante.
- Libres: Permiten usar, copiar, modificar y redistribuir el software sin restricciones. Las principales licencias libres son GNU/GPL, BSD, MIT, Apache. Ejemplo: GNU/Linux, LibreOffice. Ventaja: libertad y comunidad; desventaja: el usuario es responsable del soporte.
- Open Source (código abierto): Permite acceder, modificar y distribuir el código fuente, pero puede imponer límites a la redistribución o uso comercial. Ejemplo: Mozilla Firefox, Android. Suele ofrecer mayor transparencia y seguridad, ya que el código es revisado públicamente.
- Las licencias mixtas: son modelos que combinan elementos de licencias abiertas (open source, libres) y privativas, o que varían ciertos derechos y restricciones para diferentes partes del software o de sus usuarios. Este tipo de licencia permite adaptar el software a necesidades comerciales y de colaboración, combinando ventajas de ambos mundos.

Características clave de las licencias mixtas

- Permiten que una parte del software sea open source (código abierto y modificable) y otra privativa (código cerrado y restringido).
- Pueden ofrecer el núcleo del programa bajo licencia libre, pero añadir módulos, complementos o componentes bajo diferentes tipos de licencia, por ejemplo, privativos o comerciales.
- Suelen ofrecer diferentes niveles de acceso y funciones según el tipo de usuario, comprador o mercado: versión gratuita con funciones limitadas y versión de pago con funciones premium.
- Se utilizan mucho en aplicaciones empresariales y modelos SaaS, donde se combina el acceso libre a una plataforma básica y módulos avanzados únicamente con licencia comercial o suscripción.

- Facilitan la adopción por parte de comunidades y empresas, permitiendo contribuciones en parte del proyecto sin renunciar a derechos exclusivos sobre otros sectores del software.

Ejemplo rápido

Muchos programas de gestión, seguridad y CRM (como Elasticsearch, Oracle MySQL, Red Hat) ofrecen la versión comunitaria bajo licencia open source, pero otras versiones comerciales incluyen funcionalidades adicionales que requieren licencia de pago. Esto permite a los desarrolladores y empresas elegir el modelo de uso y distribución que más les convenga, y al proveedor financiar el desarrollo.

Ventajas y limitaciones

- Permiten flexibilidad técnica y comercial, adaptándose tanto a desarrolladores independientes como a grandes empresas.
- Es fundamental revisar cuidadosamente las condiciones legales, pues los derechos, restricciones y obligaciones pueden variar para cada componente o usuario.
- Pueden fomentar la innovación, pero requieren una gestión rigurosa para evitar incompatibilidades legales y confusiones entre usuarios y desarrolladores.